

10.8 RECOMMENDED READING

[WEND89] is an interesting discussion of approaches to multiprocessor scheduling. [ZHUR12] provides a comprehensive survey of issues related to multicore thread scheduling. The following collections of papers all contain important articles on real-time operating systems and scheduling: [KRIS94], [STAN93], [LEE93], and [TILB91]. [SHA90] provides a good explanation of priority inversion, priority inheritance, and priority ceiling. [ZEAD97] analyzes the performance of the SVR4 real-time scheduler. [LIND04] provides an overview of the Linux 2.6 scheduler; [LOVE10] contains a more detailed discussion.

KRIS94 Krishna, C., and Lee, Y., eds. "Special Issue on Real-Time Systems." *Proceedings of the IEEE*, January 1994.

LEE93 Lee, Y., and Krishna, C., eds. *Readings in Real-Time Systems*. Los Alamitos, CA: IEEE Computer Society Press, 1993.

LIND04 Lindsley, R. "What's New in the 2.6 Scheduler." *Linux Journal*, March 2004.

LOVE10 Love, R. *Linux Kernel Development*. Upper Saddle River, NJ: Addison-Wesley, 2010.

SHA90 Sha, L.; Rajkumar, R.; and Lehoczky, J. "Priority Inheritance Protocols: An Approach to Real-Time Synchronization." *IEEE Transactions on Computers*, September 1990.

STAN93 Stankovic, J., and Ramamritham, K., eds. *Advances in Real-Time Systems*. Los Alamitos, CA: IEEE Computer Society Press, 1993.

TILB91 Tilborg, A., and Koob, G., eds. *Foundations of Real-Time Computing: Scheduling and Resource Management*. Boston: Kluwer Academic Publishers, 1991.

WEND89 Wendorf, J.; Wendorf, R.; and Tokuda, H. "Scheduling Operating System Processing on Small-Scale Microprocessors." *Proceedings, 22nd Annual Hawaii International Conference on System Science*, January 1989.

ZEAD97 Zeadally, S. "An Evaluation of the Real-Time Performance of SVR4.0 and SVR4.2." *Operating Systems Review*, January 1977.

ZHUR12 Zhuravlev, S., et al. "Survey of Scheduling Techniques for Addressing Shared Resources in Multicore Processors." *ACM Computing Surveys*, November 2012.

10.9 KEY TERMS, REVIEW QUESTIONS, AND PROBLEMS

Key Terms

aperiodic task deadline scheduling deterministic operating system fail-soft operation gang scheduling granularity	hard real-time task load sharing periodic task priority inversion rate monotonic scheduling real-time operating system	real-time scheduling responsiveness soft real-time task thread scheduling unbounded priority inversion
--	---	---

Review Questions

- 10.1. List and briefly define five different categories of synchronization granularity.
- 10.2. List and briefly define four techniques for thread scheduling.
- 10.3. List and briefly define three versions of load sharing.
- 10.4. What is the difference between hard and soft real-time tasks?
- 10.5. What is the difference between periodic and aperiodic real-time tasks?
- 10.6. List and briefly define five general areas of requirements for a real-time operating system.
- 10.7. List and briefly define four classes of real-time scheduling algorithms.
- 10.8. What items of information about a task might be useful in real-time scheduling?

Problems

- 10.1. Consider a set of three periodic tasks with the execution profiles of Table 10.7. Develop scheduling diagrams similar to those of Figure 10.5 for this set of tasks.
- 10.2. Consider a set of five aperiodic tasks with the execution profiles of Table 10.8. Develop scheduling diagrams similar to those of Figure 10.6 for this set of tasks.
- 10.3. Least laxity first (LLF) is a real-time scheduling algorithm for periodic tasks. Slack time, or laxity, is the amount of time between when a task would complete if it started now and its next deadline. This is the size of the available scheduling window. Laxity can be expressed as

$$\text{Laxity} = (\text{deadline time}) - (\text{current time}) - (\text{processor time needed})$$

LLF selects the task with the minimum laxity to execute next. If two or more tasks have the same minimum laxity value, they are serviced on a FCFS basis.

Table 10.7 Execution Profile for Problem 10.1

Process	Arrival Time	Execution Time	Ending Deadline
A(1)	0	10	20
A(2)	20	10	40
•	•	•	•
•	•	•	•
•	•	•	•
B(1)	0	10	50
B(2)	50	10	100
•	•	•	•
•	•	•	•
•	•	•	•
C(1)	0	15	50
C(2)	50	15	100
•	•	•	•
•	•	•	•
•	•	•	•

Table 10.8 Execution Profile for Problem 10.2

Process	Arrival Time	Execution Time	Starting Deadline
A	10	20	100
B	20	20	30
C	40	20	60
D	50	20	80
E	60	20	70

- a. Suppose a task currently has a laxity of t . By how long may the scheduler delay starting this task and still meet its deadline?
 - b. Suppose a task currently has a laxity of 0. What does this mean?
 - c. What does it mean if a task has negative laxity?
 - d. Consider a set of three periodic tasks with the execution profiles of Table 10.9a. Develop scheduling diagrams similar to those of Figure 10.5 for this set of tasks that compare rate monotonic, earliest-deadline first, and LLF. Assume preemption may occur at 5-ms intervals. Comment on the results.
- 10.4.** Repeat Problem 10.3d for the execution profiles of Table 10.9b. Comment on the results.
- 10.5.** Maximum urgency first (MUF) is a real-time scheduling algorithm for periodic tasks. Each task is assigned an urgency that is defined as a combination of two fixed priorities and one dynamic priority. One of the fixed priorities, the criticality, has precedence over the dynamic priority. Meanwhile, the dynamic priority has precedence over the other fixed priority, called the user priority. The dynamic priority is inversely proportional to the laxity of a task. MUF can be explained as follows. First, tasks are ordered from shortest to longest period. Define the critical task set as the first N tasks such that worst-case processor utilization does not exceed 100%. Among critical set tasks that are ready, the scheduler selects the task with the least laxity. If no critical set tasks are ready, the schedule chooses among the noncritical tasks the one with the least laxity. Ties are broken through an optional user priority and then by FCFS. Repeat Problem 10.3d, adding MUF to the diagrams. Assume that user-defined priorities are A highest, B next, C lowest. Comment on the results.
- 10.6.** Repeat Problem 10.4, adding MUF to the diagrams. Comment on the results.

Table 10.9 Execution Profiles for Problems 10.3 through 10.6

(a) Light load		
Task	Period	Execution Time
A	6	2
B	8	2
C	12	3
(b) Heavy load		
Task	Period	Execution Time
A	6	2
B	8	5
C	12	3

10.7. This problem demonstrates that although Equation (10.2) for rate monotonic scheduling is a sufficient condition for successful scheduling, it is not a necessary condition (i.e., sometimes successful scheduling is possible even if Equation (10.2) is not satisfied).

a. Consider a task set with the following independent periodic tasks:

- **Task P₁:** $C_1 = 20$; $T_1 = 100$
- **Task P₂:** $C_2 = 30$; $T_2 = 145$

Can these tasks be successfully scheduled using rate monotonic scheduling?

b. Now add the following task to the set:

- **Task P₃:** $C_3 = 68$; $T_3 = 150$

Is Equation (10.2) satisfied?

c. Suppose that the first instance of the preceding three tasks arrives at time 0. Assume that the first deadline for each task is the following:

$$D_1 = 100; \quad D_2 = 145; \quad D_3 = 150$$

Using rate monotonic scheduling, will all three deadlines be met? What about deadlines for future repetitions of each task?

10.8. Draw a diagram similar to that of Figure 10.9b that shows the sequence events for this same example using priority ceiling.